CAHIERS DE LINGUISTIQUE ASIE ORIENTALE
EAST ASIAN LANGUAGES AND LINGUISTICS
October 13, 2026

From Paper to Digital

A Java-Based Order Management System for a Local Cafe

Gavin G Waako
Nano Software Solutions
g_waako@massbay.edu

Abstract

Small independent cafes often take orders on paper, which is slow and error-prone during peak hours. This paper presents the design of a Java desktop point of sale (POS) system built for a local cafe in Wellesley, Massachusetts, whose staff took orders and calculated totals by hand during the 7 AM to 10 AM morning rush. The system follows the Model-View-Controller (MVC) architecture and is delivered in two phases: a Java Swing prototype that combines the menu and a live cart in a single keyboard-driven dialog, and a planned JavaFX interface with large, high-contrast, touch-optimized buttons. The software automates subtotal and tax calculation, shows the cart in real time, saves each sale immediately to local storage so that a power failure does not lose data, loads menu items and prices from an external data file, and uses exception handling so that invalid input produces a helpful alert instead of a crash. A planned administrator function produces an end-of-day summary so that cash and digital records can be reconciled without a manual tally.

The paper describes the requirements, architecture, interface design, risk mitigation and a 226-hour development plan, and proposes an evaluation plan with measures for order-entry time, calculation accuracy and end-of-day reconciliation. Field results are not reported here; they are left for the evaluation stage.

Keywords

point of sale – Java – Model-View-Controller – Swing – JavaFX – usability – small business – exception handling

Résumé

Les petits cafés indépendants prennent souvent les commandes sur papier, ce qui est lent et source d'erreurs aux heures de pointe. Cet article présente la conception d'un système de point de vente (POS) de bureau en Java, développé pour un café local de Wellesley, dans le Massachusetts, dont le personnel prenait les commandes et calculait les totaux à la main pendant l'affluence matinale, de 7 h à 10 h. Le système suit l'architecture Modèle-Vue-Contrôleur (MVC) et est livré en deux phases : un prototype Java Swing qui réunit le menu et un panier en direct dans une seule boîte de dialogue pilotée au clavier, puis une interface JavaFX prévue, dotée de grands boutons à fort contraste adaptés à l'écran tactile. Le logiciel automatise le calcul du sous-total et de la taxe, affiche le panier en temps réel, enregistre chaque vente immédiatement dans un stockage local afin qu'une coupure de courant n'entraîne aucune perte de données, charge les articles et les prix du menu depuis un fichier de données externe et utilise la gestion des exceptions pour qu'une saisie invalide produise une alerte utile plutôt qu'un plantage. Une fonction d'administration prévue produit un récapitulatif de fin de journée afin de rapprocher les espèces et les enregistrements numériques sans décompte manuel.

L'article décrit les besoins, l'architecture, la conception de l'interface, l'atténuation des risques et un plan de développement de 226 heures, puis propose un plan d'évaluation avec des mesures du temps de saisie des commandes, de l'exactitude des calculs et du rapprochement de fin de journée. Les résultats sur le terrain ne sont pas présentés ici ; ils sont renvoyés à l'étape d'évaluation.

Keywords

point de vente – Java – Modèle-Vue-Contrôleur – Swing – JavaFX – ergonomie – petite entreprise – gestion des exceptions

1 Introduction

A point of sale system records what a customer orders, calculates what they owe, and keeps a record of the sale. Large commercial POS products exist, but the owner of the cafe in this study reported that the corporate software firms she consulted did not understand the rhythm of a small, local cafe. Her business needed something simpler, built around how her own staff work.

The cafe's difficulty is concentrated in a single window of the day. During the morning peak from 7 AM to 10 AM the line often reaches the door. Orders were taken on a paper pad, and totals and taxes were calculated mentally. The owner described steaming milk with one hand while calculating an order total on paper with the other. Under that pressure, taxes were sometimes miscalculated and food modifications, such as warming a muffin, were forgotten. The staff reported the same struggle: they are baristas, not mathematicians, and the fatigue of constant manual tallying leads to errors that hurt customer service. At the end of the day, physical cash had to be matched against paper records by hand.

The goal of this project is therefore to replace the paper process with a custom Java desktop application that does the arithmetic and the record keeping, so that staff can concentrate on the quality of the drinks. The scope includes a simple user interface, an automatic calculator for prices and taxes, persistent storage of every sale, and, in future updates, receipt generation and an administrator report at the close of business.

The contributions of this paper are:

1. A set of functional and non-functional requirements derived directly from the cafe owner's description of her workflow.
2. An MVC design for a small order management system, with a worked example of its pricing and tax computation.
3. A two-phase interface strategy: a Swing prototype for rapid delivery, followed by a touch-optimized JavaFX interface.
4. A risk analysis and a 226-hour development plan for a one-developer project.
5. A proposed evaluation plan for measuring the benefit of the system once it is in use.

The rest of the paper is organized as follows. Section 2 reviews related concepts. Section 3 states the requirements. Section 4 describes the system design. Section 5 describes the user interface. Section 6 analyzes risks. Section 7 gives the development plan. Section 8 presents the evaluation plan and expected results. Sections 9 and 10 conclude and outline further enhancements.

2 Literature Review

This project draws on established ideas from human-computer interaction and software architecture. This section summarizes the ones that shaped the design.

Interaction under time pressure. Fitts's law (**fitts1954**) models the time needed to acquire a target as a function of its distance and its size: larger targets that are closer to the hand are reached faster and with fewer errors. This is the basis for the large buttons planned for the touch interface. Hick's law (**hick1952**) describes how the time to choose between alternatives grows with the number of alternatives, which supports keeping the menu short and the number of steps per order small. Cognitive load theory (**sweller1988**) explains why mental arithmetic is costly for a person who is also doing a physical task: working memory is limited, so removing the calculation frees capacity for the primary task of preparing drinks.

Usability principles. Nielsen's usability heuristics (**nielsen1993**) include visibility of system status, error prevention, and recognition rather than recall. These map directly onto the cafe's needs: a live cart makes the state of the order visible at all times, automatic calculation prevents arithmetic errors, and a menu with numbered or pictured items lets staff recognize an item instead of remembering it.

Architecture. The Model-View-Controller paradigm (**krasner1988**) separates the data and rules of an application (the model) from how they are displayed (the view) and from the logic that connects them (the controller). Gamma et al. (**gamma1994**) describe MVC as a composition of well-known object-oriented patterns. For this project the separation has a practical payoff: the Swing view can be replaced by a JavaFX view in a later phase without rewriting the pricing and order logic.

Robustness. Bloch (**bloch2018**) recommends checked exceptions for conditions from which a caller can reasonably recover, and unchecked exceptions for programming errors. Invalid user input is a recoverable condition, so the system defines its own checked exception for invalid menu choices and turns it into a message for the barista.

Hardware integration. The Java Print Service API (**oracleprint**) allows a Java program to discover printers and send them print jobs. It is the standard route to a thermal receipt printer without

[width=0.75]Use Case Diagram.jpg
FIGURE 1: *Enter Caption*

vendor-specific software.

Together, these sources suggest a design that is simple at the point of interaction, strict about calculation, and modular underneath.

3 Problem Analysis and Requirements

3.1 *Context*

The system is used at one location by a small team: baristas who take orders, and the owner, who manages the menu and reviews daily totals. The customer is a third party who interacts only with the barista. These roles, and the actions available to each, are shown in the use case diagram.

[Figure placeholder: Fig. 1. Use case diagram of the MilkyBay Cafe POS system, with actors Barista, Customer and Administrator (owner).]

The Barista can view and browse the menu, view the live cart, add products to the cart, clear the cart, initiate payment and exit the program. The Administrator signs in or registers and then has additional privileges: printing the daily report, adding new products to the menu, and editing products on the menu.

3.2 *Functional requirements*

3.3 *Non-functional requirements*

4 System Design

4.1 *MVC architecture*

The application is organized into four packages: a model package (`com.cafe.model`), a view package (`com.cafe.view`), a controller package (`com.cafe.controller`) and an entry point (`com.cafe.main`). The entry point creates one instance each of the model, view and controller and starts the controller.

[Figure placeholder: Fig. 2. Class diagram of the MilkyBay Cafe POS using MVC architecture.]

TABLE 1: *Functional requirements*

ID	Requirement	Source
F1	Display the menu with item names and prices	Use case: browse menu
F2	Add an item to the cart with a single action	Owner request: one-tap design
F3	Show a live cart with subtotal and tax that updates instantly	Owner request
F4	Record simple item modifications, such as warming a muffin	Owner description of lost modifications
F5	Clear the cart	Use case
F6	Initiate payment and complete the sale	Use case
F7	Save every sale persistently	Project overview
F8	Generate a daily summary report for the administrator (future update)	Owner request
F9	Allow the administrator to add and edit menu items	Seasonal menu changes
F10	Print a receipt (future update)	Project scope

TABLE 2: *Non-functional requirements*

ID	Requirement
N1	Usability: an order can be finalized in one or two taps; one-handed operation
N2	Reliability: no loss of transaction data on sudden power loss
N3	Stability: the program stays functional when input is rapid or unexpected
N4	Maintainability: menu items and prices can change without editing source code
N5	Platform: Windows 10 or 11, Java JDK 21, 15-inch or larger touch screen, thermal receipt printer, at least 8 GB of RAM

TABLE 3: *Main components and their responsibilities*

Component	Responsibility
`Product` (interface)	Holds a name and a price; exposes `getName`, `getPrice` and `getDescription`
`Drink`, `Food`	Concrete products; each supplies its own description
`OrderInterface`	Declares `processTransaction`
`OrderManager`	Holds the cart as a list of products; builds the menu text, computes the subtotal, adds items, clears the cart and saves sales
`InvalidChoiceException`	Checked exception, a subclass of `Exception`, raised for an invalid menu choice
`CafeGUI` (view)	Displays the menu and cart and shows messages, using Swing's `JOptionPane`
`CafeController`	Holds references to the model and the view; starts the application, handles each input and builds the cart text

The separation keeps all arithmetic and order state in the model. The view only displays text it is given, and the controller translates each input into a model operation. A product hierarchy with `Drink` and `Food` as subclasses of `Product` allows each type to describe itself differently and lets new categories be added without changing the cart logic.

4.2 *Pricing and tax computation*

Let the cart contain n lines. Line i has unit price p_i and quantity q_i. The subtotal S, the tax T and the total are computed as:

$$S = \sum_{i=1}^{n} p_i \times q_i \tag{1}$$

$$T = \text{round}(r \times S,\, 2) \tag{2}$$

$$\text{Total} = S + T \tag{3}$$

where r is the tax rate and $\text{round}(x, 2)$ rounds to two decimal places (half up). The rate is held as a configurable value; the prototype interface uses $r = 0.07$. All three values are recomputed whenever the cart changes, so the barista never performs the calculation.

As a worked example, take the cart shown in the interface mockup: one Large Latte at $4.50, one Muffin (warmed) at $3.00, and two Breakfast Burritos for a line total of $11.00. By Eq. (1), $S = 4.50 + 3.00 + 11.00 = \$18.50$. By Eq. (2), $T = \text{round}(0.07 \times 18.50, 2) = \text{round}(1.295, 2) = \$1.30$. By Eq. (3), the total is $19.80, which matches the amount shown on screen.

4.3 *Persistence and data-driven menu*

Two kinds of data must outlive a single session: the menu and the sales. Sales are written to storage as soon as a payment is completed, rather than at the end of the day, so that a sudden loss of power loses at most the cart in progress. The development plan specifies a SQLite database for sales. The menu is read from an external text file at start-up. This makes the program data-driven: when the seasonal menu changes, the owner edits a list of items and prices instead of asking a programmer to change the source code.

4.4 *Error handling*

The controller wraps each input in a handler that catches the program's own `InvalidChoiceException` as well as unexpected runtime errors. An invalid code or an empty input produces a short message from the view, and the cart is left unchanged. This is intended to keep the program running when a user types quickly, mistypes, or enters unexpected data during a rush.

5 User Interface

The interface follows a utility-first philosophy: every design choice is judged by whether it helps a person who is working fast with one hand free. Development is split into two phases so that the cafe can start using a working version early.

5.1 *Phase 1: Java Swing prototype*

The first implementation uses the Swing `JOptionPane` class. It shows a single modal dialog that combines the menu and the live cart, and it is driven by numeric codes typed on the keyboard. The barista does not need to aim a pointer at a small target; typing a code and pressing Enter is enough.

TABLE 4: *Menu codes and prices in the Swing prototype*

Code	Item	Price
1	Signature Latte	$4.50
2	English Tea	$3.00
3	Bottled Water	$2.00
4	Peach Iced Tea	$3.75
5	Blueberry Muffin	$3.00
6	Classic Beef Burger	$8.99
7	Hot Chocolate	$3.50
8	Croissant	$3.25
9	Pay	–
10	Clear	–
0	Exit	–

[Figure placeholder: Fig. 3. Swing prototype dialog showing the numbered menu and an empty cart.]

5.2 *Phase 2: JavaFX touch interface*

The second phase moves to JavaFX to deliver the high-contrast, tap-and-go design the owner asked for. The screen is a grid of large, color-coded buttons for signature items, a dedicated live-cart panel showing the subtotal, tax and total, and three large action buttons: Pay and Print Receipt, Clear Order, and View Shift Summary. The layout is sized for 15-inch touch monitors and is designed to prevent mis-taps during peak hours, following Fitts's law (**fitts1954**).

[Figure placeholder: Fig. 4. JavaFX interface mockup with item grid, live cart and action buttons.]

5.3 *Design principles*

Both versions follow the same principles: high contrast, minimal cognitive load (**sweller1988**), visible system status through the live cart (**nielsen1993**), automatic calculation so that no mental math is needed, and clear alerts instead of crashes when input is invalid.

6 Risk Analysis and Mitigation

A small project in a live business carries risks that are as much about people and equipment as about code. Table 5 lists the main

risks identified and the planned response.

TABLE 5: *Risks and planned mitigation*

Risk	Category	Planned mitigation
Staff find the software slow or confusing at rush hour	Usability	Minimalist design; one or two taps to finalize an order
Data lost if power fails	Reliability	Save transactions immediately to local storage
Older printers or peripherals conflict with the software	Technical	Use the standard Java Print API (**oracleprint**); test hardware compatibility early
GUI fails when used too fast or with unexpected input	Technical	Robust exception handling (**bloch2018**); keep the program running on bad input
Baristas prefer the familiar paper system	Market	Use the owner's direct feedback; match button layout to the barista's natural workflow
Seasonal menu changes	Market	Data-driven menu in an external text file
Hardware prices rise before purchase	Budget	Provide a finalized hardware list immediately so equipment is bought at current prices
Scope creep raises labor cost	Budget	Fixed schedule; prioritize essential requirements within 226 hours

7 Development Plan and Effort

7.1 *Phases*

The plan runs from January to June 2026 and has five phases: analysis, development, testing, deployment and training. The estimated total effort is 226 hours.

Testing is the largest phase at 92 hours, about 41 percent of the total, reflecting the priority on reliability in a live setting. Milestones with a reporting requirement, such as the interface review with the owner, are used as checkpoints for client feedback.

7.2 *Hardware and software*

The system is designed for a desktop terminal or laptop running Windows 10 or 11, with a 15-inch or larger touch screen, a standard

TABLE 6: *Development phases and estimated effort*

Phase	Main tasks	Hours
1. Analysis	Visit the cafe and draw a first mockup; decide how classes and objects work together; plan the division of work	34
2. Development	Set up the SQLite database; load menu items and prices; correct menu data errors; build the button interface; connect the printing system; send reports to a local computer	65
3. Testing	Alpha test to find major bugs; test additional features for stability; beta test with volunteer workers in the cafe; write the user manual	92
4. Deployment	Install on the cafe computers; install and check the connection for management	15
5. Training	Train the owner and baristas; support the staff after the first day	20
Total		**226**

thermal receipt printer connected by USB or local network, and at least 8 GB of RAM so that the Java environment stays responsive in busy periods. The software is written in Java (JDK 21), using Swing for the first interface and JavaFX for the planned touch interface, with IntelliJ IDEA as the development environment.

8 Evaluation Plan and Expected Results

8.1 *Evaluation plan*

The development plan schedules an alpha test to find major bugs and a beta test in which volunteer cafe workers use the system during real service. To judge whether the system achieves its purpose, the following measures are proposed. They extend the charter and are intended to be collected during the beta test and the first weeks of use.

8.2 *Expected results*

Some outcomes follow from the design and can be stated in advance. Because subtotal, tax and total are computed by Eq. (1) to (3), arithmetic errors by staff are removed as a source of error; the

TABLE 7: *Proposed evaluation measures*

Measure	How it is collected	Comparison
Order-entry time per transaction	Timestamps recorded by the system, per order	Paper process timed during a sample of peak periods
Taps or keystrokes per order	Counted from the interaction log	Design goal of one or two taps to finalize
Calculation accuracy	Totals checked against an independent reference calculation	Hand-calculated totals from the paper process
End-of-day reconciliation	Difference between cash counted and system total	Difference under the paper process
Staff acceptance	Short structured feedback from baristas and the owner	Not applicable

remaining risk is an incorrect price in the menu file, which is why the plan includes a task to correct menu data. Because each sale is saved at the moment of payment, the end-of-day total can be produced by the system rather than tallied from paper, which is the basis for aligning cash with digital records. Improvements in service speed and in staff experience are expected, but they are hypotheses to be tested with the measures above, not results.

8.3 *Limitations*

The study concerns one cafe and one development team, so the findings will not necessarily generalize. The prototype represents prices as floating-point values, which can introduce rounding differences in more complex calculations; Section 10 proposes a safer representation. Finally, the Swing prototype relies on typed codes, which suits keyboard use but not the touch hardware that the final deployment targets.

9 Conclusion

This paper presented the design of a Java-based order management system for a local cafe whose morning rush exposed the limits of a paper process. The system uses an MVC architecture so that the pricing and order logic is independent of the interface, delivers

a working Swing prototype first and a touch-optimized JavaFX interface later, automates subtotal and tax calculation, saves sales immediately, keeps the menu in an external file, and handles invalid input without crashing. A risk analysis covering usability, technical, market and budget concerns, and a 226-hour plan, show how a single developer can deliver the system within a fixed budget. The benefits for service speed, accuracy and reconciliation are expected from the design but remain to be measured, and the evaluation plan in Section 8 describes how.

10 Further Enhancements

As future work, the following points will be considered:

- Completing the JavaFX interface with color-coded item buttons, a visual cart and a shift summary view.
- Generating and printing receipts through the Java Print Service (**oracleprint**).
- An administrator mode with sign-in, a daily summary report and menu editing.
- Recording item modifications, such as warming a muffin, on the order line and on the receipt.
- Representing money with `BigDecimal` or integer cents instead of floating-point numbers.
- A management view that lets the owner check daily totals away from the counter.
- Reporting measured results from the beta test and the first weeks of use.

Acknowledgments

The author thanks the owner and staff of the cafe for their time, description of their workflow and feedback on the interface design.

References

Boltz, William G. & Shapiro, Michael C. (eds.). 1991. *Studies in the historical phonology of Asian languages*. Amsterdam: John Benjamins. https://doi.org/10.1075/cilt.77.

Haudricourt, André-Georges. 1954. De l'origine des tons en vietnamien. *Journal Asiatique* 242. 69–82.

Martin, Samuel E. 1975. *A reference grammar of Japanese*. New Haven: Yale University Press.

Ramstedt, Gustaf J. 1903. *Über die Konjugation des Khalkha-Mongolischen* (Mémoires de la Société Finno-Ougrienne 19). Helsinki: Finnischen Litteraturgesellschaft.